

Propositional Logic

Ole C. Eidheim

September 17, 2026

Department of Computer Science

Outline

Introduction

Propositional logic

Inference rules

Horn and definite clauses

Resolution

Example application

“**Reason** is the capacity to consciously apply **logic** by drawing valid conclusions from new or existing information, with the aim of seeking truth.”

“**Reason** is the capacity to consciously apply **logic** by drawing valid conclusions from new or existing information, with the aim of seeking truth.”

. . .

“In artificial intelligence and computer science, scientists study and use **automated reasoning** for diverse applications. . .”

AtB's previous natural language service: bussorakelet (BusTUC)



Presse | FAQ | English

06:59

StartsideRuteinformasjonBilletter og priserReiseinformasjonOm AtBKontakt oss

Du er her: Startside

Søkebegrep

Min reisekonto

Logg inn

Ny reisekonto

t:kort

Bestill t:kort

Del din idé

Send oss ideer på hva du ønsker å se på det nye nettstedet.

► Send inn ditt forslag

Kontakt oss

Adresse: Kongensgt. 34

Telefon: 02820

E-post: atb@atb.no

► Kontaktskjema

Reiseplanlegger

Vi tar dæ me' i Sør-Trøndelag

fra A

te B

6

45

03.09.2010

☐ Avgang etter

☐ Ankomst før

Eller spør bussorakelet (gjelder Trondheim) [Les mer](#)

Når går bussen...

▶

▶

Someth went wrong :(



Holdeplasser

Viser nå den tidligste passering av holdeplassen du står på.

▶

Aktuelt



Vinnerne er kåret

02.09.2010

Vinnerne av AtB konkurransen vi hadde i august er kåret. Tre personer vinner et verdikort på kr 1.000 hver. I tillegg har vi kåret vinneren av barnekonkurransen, fargelegg din AtB-buss.

[Les mer](#) ▶

Natural language demo (not curriculum)

Propositional logic agent code example

Example dialog:

Hello! I am a propositional logic agent.

> If it is raining then it is wet.

I did not know that. The knowledge base has been updated.

> Is it raining?

I do not know.

> It is raining.

I did not know that. The knowledge base has been updated.

> Is it raining?

Yes.

> Is it wet?

Yes.

> Is it slippery?

I do not know.

> It is slippery if it is wet.

I did not know that. The knowledge base has been updated.

> Is it slippery?

Yes.

Natural language is hard to interpret

- A dime is better than a nickel
- A nickel is better than a penny

Therefore, a dime is better than a penny

Natural language is hard to interpret

- A dime is better than a nickel
- A nickel is better than a penny

Therefore, a dime is better than a penny

- A penny is better than nothing
- Nothing is better than world peace

Therefore, a penny is better than world peace

Outline

Introduction

Propositional logic

Inference rules

Horn and definite clauses

Resolution

Example application

Logical connectives used in the logic lectures

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

Example with implies $\Rightarrow$, i.e. a conditional sentence:

the polygon is a square $\Rightarrow$ the polygon is a rectangle

Example with if and only if $\Leftrightarrow$, i.e. a biconditional sentence:

the polygon is a rectangle $\Leftrightarrow$ the polygon has four edges and four equal angles

Definitions

- Proposition symbol: a symbol that is *true* or *false*, e.g. $Rain \in \{true, false\}$
 - Denoted by capital first letter, e.g. $P, Q, Rain$
- Sentence: a statement that may be *true* or *false*, e.g. $Rain \wedge Wind$
 - Denoted by Greek letters, e.g. α, β, γ
- Model: specific symbol assignments, e.g. $\{Rain = true, Wind = false\}$
- Knowledge base (KB): set of sentences that represent truths, e.g.
 $KB = \{Rain, Rain \Rightarrow Wet\}$, i.e. it is raining, and rain implies wet

Knowledge base, continued

A knowledge base KB is sometimes used as a sentence. The KB is then treated as a conjunction of all the sentences in the KB set.

For example, given:

$$KB = \{Rain, Rain \Rightarrow Wet, Wet \Rightarrow Slippery\}$$

then:

$$KB \vee Wind = (Rain \wedge (Rain \Rightarrow Wet) \wedge (Wet \Rightarrow Slippery)) \vee Wind$$

I.e. the sentences in the KB are all *true* $\Leftrightarrow$ the conjunction of the sentences in the KB is *true*

Satisfaction, models, entailment, and model checking

Satisfaction

If a sentence α is *true* in model (symbol assignment) m , then m satisfies α

Example: Model $\{Rain = true, Wind = false\}$ satisfies sentence $Rain \wedge \neg Wind$

Satisfaction, models, entailment, and model checking

Satisfaction

If a sentence α is *true* in model (symbol assignment) m , then m satisfies α

Example: Model $\{Rain = true, Wind = false\}$ satisfies sentence $Rain \wedge \neg Wind$

Models of a sentence

$M(\alpha)$: set of all models (symbol assignments) that satisfies sentence α

Examples:

$$M(Rain) = \{\{Rain = true, Wind = false\}, \{Rain = true, Wind = true\}\}$$

$$M(Rain \vee Wind) = \{\{Rain = false, Wind = true\}, \\ \{Rain = true, Wind = false\}, \{Rain = true, Wind = true\}\}$$

Satisfaction, models, entailment, and model checking

Satisfaction

If a sentence α is *true* in model (symbol assignment) m , then m satisfies α

Example: Model $\{Rain = true, Wind = false\}$ satisfies sentence $Rain \wedge \neg Wind$

Models of a sentence

$M(\alpha)$: set of all models (symbol assignments) that satisfies sentence α

Examples:

$$M(Rain) = \{\{Rain = true, Wind = false\}, \{Rain = true, Wind = true\}\}$$

$$M(Rain \vee Wind) = \{\{Rain = false, Wind = true\}, \\ \{Rain = true, Wind = false\}, \{Rain = true, Wind = true\}\}$$

Entailment

$\alpha \models \beta$: the sentence α entails the sentence β if and only if $M(\alpha) \subseteq M(\beta)$

Closely related to the $\Rightarrow$ connective: $(\alpha \models \beta) \Leftrightarrow (\alpha \Rightarrow \beta \text{ is } true \text{ for all models})$

Example: $Rain \models Rain \vee Wind$

Satisfaction, models, entailment, and model checking

Satisfaction

If a sentence α is *true* in model (symbol assignment) m , then m satisfies α

Example: Model $\{Rain = true, Wind = false\}$ satisfies sentence $Rain \wedge \neg Wind$

Models of a sentence

$M(\alpha)$: set of all models (symbol assignments) that satisfies sentence α

Examples:

$$M(Rain) = \{\{Rain = true, Wind = false\}, \{Rain = true, Wind = true\}\}$$

$$M(Rain \vee Wind) = \{\{Rain = false, Wind = true\}, \\ \{Rain = true, Wind = false\}, \{Rain = true, Wind = true\}\}$$

Entailment

$\alpha \models \beta$: the sentence α entails the sentence β if and only if $M(\alpha) \subseteq M(\beta)$

Closely related to the $\Rightarrow$ connective: $(\alpha \models \beta) \Leftrightarrow (\alpha \Rightarrow \beta \text{ is } true \text{ for all models})$

Example: $Rain \models Rain \vee Wind$

Model checking: checks all possible models if $\alpha \Rightarrow \beta$ is *true* (infeasible in practice)

Proving entailment using inference rules

Proving $Rain \models Rain \vee Wind$ by showing that $Rain \Rightarrow Rain \vee Wind$ is always *true* using the inference rule *or introduction* and Lean proof assistant (not curriculum):

```
example ( Rain Wind : Prop ) : Rain → Rain ∨ Wind := by
  intro r
  -- Assume Rain (as assumption r),
  -- then we must prove Rain ∨ Wind
  have rw : Rain ∨ Wind := Or.inl r -- If Rain is true,
  -- then Rain ∨ Wind is also true
  exact rw -- Goal is exactly rw
```

Check for unsatisfiability (proof by contradiction)

Proving entailment by checking for unsatisfiability, i.e. finding contradiction in $\alpha \wedge \neg\beta$:

$$(\alpha \models \beta) \Leftrightarrow (M(\alpha \wedge \neg\beta) = \emptyset)$$

$$(\alpha \models \beta) \Leftrightarrow (\alpha \wedge \neg\beta \text{ is } \textit{false} \text{ for all models})$$

Check for unsatisfiability (proof by contradiction)

Proving entailment by checking for unsatisfiability, i.e. finding contradiction in $\alpha \wedge \neg\beta$:

$$(\alpha \models \beta) \Leftrightarrow (M(\alpha \wedge \neg\beta) = \emptyset)$$

$$(\alpha \models \beta) \Leftrightarrow (\alpha \wedge \neg\beta \text{ is } \textit{false} \text{ for all models})$$

$$(\alpha \Rightarrow \beta \text{ is } \textit{true} \text{ for all models}) \Leftrightarrow (\neg(\alpha \wedge \neg\beta) \text{ is } \textit{true} \text{ for all models})$$

Check for unsatisfiability (proof by contradiction)

Proving entailment by checking for unsatisfiability, i.e. finding contradiction in $\alpha \wedge \neg\beta$:

$$(\alpha \models \beta) \Leftrightarrow (M(\alpha \wedge \neg\beta) = \emptyset)$$

$$(\alpha \models \beta) \Leftrightarrow (\alpha \wedge \neg\beta \text{ is } \textit{false} \text{ for all models})$$

$$(\alpha \Rightarrow \beta \text{ is } \textit{true} \text{ for all models}) \Leftrightarrow (\neg(\alpha \wedge \neg\beta) \text{ is } \textit{true} \text{ for all models})$$

Proof in Lean for $(\alpha \Rightarrow \beta) \equiv \neg(\alpha \wedge \neg\beta)$ (not curriculum):

```
example (α β : Prop) : (α → β) ↔ ¬(α ∧ ¬β) := by
  constructor
  intro ab
  rw [not_and]
  rw [not_not]
  exact ab
  -- Prove both directions of ↔, → first
  -- Assume α → β, then prove ¬(α ∧ ¬β)
  -- Use rule ¬(a ∧ b) ≡ a → ¬b
  -- Use rule ¬¬a ≡ a
  -- Proof done, now prove the other direction
  intro ab
  -- Assume ¬(α ∧ ¬β), then prove α → β
  -- Use rule a → b ≡ ¬a ∨ b
  -- Use rule a ∨ b ≡ ¬(¬a ∧ ¬b)
  -- Use rule ¬¬a ≡ a
  -- Proof done
```

Check for unsatisfiability example using z3 (not curriculum)

```
from z3 import *

set_param("proof", True)

# Symbols
Rain, Wind = Bools("Rain Wind")

s = Solver()

s.add(Rain) # Add assumption

s.add(Not(Or(Rain, Wind))) # Add negated goal  $\neg(\text{Rain} \vee \text{Wind})$ 

result = s.check() # Check  $\text{Rain} \models \text{Rain} \vee \text{Wind}$ 

print(result)
if result == sat: # A model was found that satisfies the KB
    print(s.model()) # Show model
elif result == unsat: # A contradiction was found
    print(s.proof()) # Show proof

# Output:
# unsat
# unit-resolution(asserted(Rain), ...)
```

Knowledge-based agents

$$ask(KB, \alpha) = \begin{cases} \text{yes,} & \text{if } KB \models \alpha \\ \text{no,} & \text{if } KB \models \neg\alpha \\ \text{I don't know,} & \text{otherwise} \end{cases}$$

$$tell(KB, \alpha) = \begin{cases} \text{I already know,} & \text{if } KB \models \alpha \\ \text{that can't be,} & \text{if } KB \models \neg\alpha \\ \text{I'll add to KB,} & \text{otherwise} \end{cases}$$

Examples, given $KB = \{Rain\}$:

$$ask(KB, Rain \vee Wind) = \text{yes}$$

$$ask(KB, \neg Rain) = \text{no}$$

$$ask(KB, Wind) = \text{I don't know}$$

Outline

Introduction

Propositional logic

Inference rules

Horn and definite clauses

Resolution

Example application

Logical equivalence

Logical equivalence $\equiv$ vs if and only if $\Leftrightarrow$

$(\alpha \equiv \beta) \Leftrightarrow (\alpha \Leftrightarrow \beta \text{ is } \textit{true} \text{ for all models})$

Logical equivalence $\equiv$ vs entailment $\models$

$(\alpha \equiv \beta) \Leftrightarrow (\alpha \models \beta) \wedge (\beta \models \alpha)$

Examples:

$$\alpha \equiv \alpha \wedge \alpha$$

$$\alpha \equiv \alpha \vee \alpha$$

$$\alpha \wedge \neg \alpha \equiv \textit{False} \quad \text{contradiction}$$

$$\alpha \vee \neg \alpha \equiv \textit{True} \quad \text{tautology/valid sentence}$$

$$\alpha \wedge \beta \equiv \textit{True} \wedge \alpha \wedge \beta$$

$$\alpha \vee \beta \equiv \textit{False} \vee \alpha \vee \beta$$

Logical equivalences

$$\alpha \wedge \beta \equiv \beta \wedge \alpha$$

commutativity of $\wedge$

$$\alpha \vee \beta \equiv \beta \vee \alpha$$

commutativity of $\vee$

$$(\alpha \wedge \beta) \wedge \gamma \equiv \alpha \wedge (\beta \wedge \gamma)$$

associativity of $\wedge$

$$(\alpha \vee \beta) \vee \gamma \equiv \alpha \vee (\beta \vee \gamma)$$

associativity of $\vee$

$$\neg(\neg\alpha) \equiv \alpha$$

double-negation elimination

$$\alpha \Rightarrow \beta \equiv \neg\beta \Rightarrow \neg\alpha$$

contraposition

$$\alpha \Rightarrow \beta \equiv \neg\alpha \vee \beta$$

implication elimination

$$\alpha \Leftrightarrow \beta \equiv (\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$$

biconditional elimination

$$\neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta$$

De Morgan

$$\neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta$$

De Morgan

$$\alpha \wedge (\beta \vee \gamma) \equiv (\alpha \wedge \beta) \vee (\alpha \wedge \gamma)$$

distributivity of $\wedge$ over $\vee$

$$\alpha \vee (\beta \wedge \gamma) \equiv (\alpha \vee \beta) \wedge (\alpha \vee \gamma)$$

distributivity of $\vee$ over $\wedge$

One can solve the following equation by searching over possible values of x :

$$2x = 4$$

or one can divide by two on both sides to reach $x = 2$ (using the division rule for equations)

In logic, in order to check $KB \models Wet$ given $KB = \{Rain, Rain \Rightarrow Wet\}$, one can apply inference rules on KB until the conclusion Wet is reached:

$$KB = \{Rain, Rain \Rightarrow Wet, Wet\}$$

Inference rule: and-elimination

And-elimination

$\alpha \wedge \beta$ is *true*, therefore α is *true* and β is *true*:

$$\frac{\alpha \wedge \beta}{\alpha}$$

$$\frac{\alpha \wedge \beta}{\beta}$$

For example, starting with:

$$KB = \{Rain \wedge Wet\}$$

Apply and-elimination to $Rain \wedge Wet$ to derive $Rain$:

$$KB = \{Rain \wedge Wet, Rain\}$$

Apply and-elimination to $Rain \wedge Wet$ to derive Wet :

$$KB = \{Rain \wedge Wet, Rain, Wet\}$$

Inference rule: Modus Ponens

Modus Ponens

α is *true* and $\alpha \Rightarrow \beta$ is *true*, therefore β must also be *true*:

$$\frac{\alpha, \quad \alpha \Rightarrow \beta}{\beta}$$

For example, starting with:

$$KB = \{Rain \Rightarrow Wet, Wet \Rightarrow Slippery, Rain\}$$

Apply Modus Ponens to *Rain* and $Rain \Rightarrow Wet$ to derive *Wet*:

$$KB = \{Rain \Rightarrow Wet, Wet \Rightarrow Slippery, Rain, Wet\}$$

Apply Modus Ponens to *Wet* and $Wet \Rightarrow Slippery$ to derive *Slippery*:

$$KB = \{Rain \Rightarrow Wet, Wet \Rightarrow Slippery, Rain, Wet, Slippery\}$$

LLMs can do some Modus Ponens

LLMs have been extensively trained on text like:

If it is raining, I cannot mow the lawn.

...

Oh no, it is raining, so I cannot mow the lawn.

The LLMs will then predict the next word based on similar patterns:

If α , then β It is α , therefore _

Soundness and completeness of inference algorithms

Derivation

$KB \vdash_i \alpha$: sentence α is derivable from KB by inference algorithm i

Example: with a Modus Ponens based algorithm (MPA): $\{\alpha, \alpha \Rightarrow \beta\} \vdash_{MPA} \beta$

Soundness and completeness of inference algorithms

Derivation

$KB \vdash_i \alpha$: sentence α is derivable from KB by inference algorithm i

Example: with a Modus Ponens based algorithm (MPA): $\{\alpha, \alpha \Rightarrow \beta\} \vdash_{MPA} \beta$

Soundness

A sound inference algorithm i can derive only sentences that are entailed:

$$KB \vdash_i \alpha \Rightarrow KB \models \alpha$$

MPA is sound, e.g. $\{\alpha, \alpha \Rightarrow \beta\} \vdash_{MPA} \beta$ and $\{\alpha, \alpha \Rightarrow \beta\} \models \beta$

Soundness and completeness of inference algorithms

Derivation

$KB \vdash_i \alpha$: sentence α is derivable from KB by inference algorithm i

Example: with a Modus Ponens based algorithm (MPA): $\{\alpha, \alpha \Rightarrow \beta\} \vdash_{MPA} \beta$

Soundness

A sound inference algorithm i can derive only sentences that are entailed:

$$KB \vdash_i \alpha \Rightarrow KB \models \alpha$$

MPA is sound, e.g. $\{\alpha, \alpha \Rightarrow \beta\} \vdash_{MPA} \beta$ and $\{\alpha, \alpha \Rightarrow \beta\} \models \beta$

Completeness

A complete inference algorithm i can at minimum derive all sentences that are entailed:

$$KB \models \alpha \Rightarrow KB \vdash_i \alpha$$

MPA is not complete since $\{\alpha, \alpha \vee \beta \Rightarrow \gamma\} \not\vdash_{MPA} \gamma$ and $\{\alpha, \alpha \vee \beta \Rightarrow \gamma\} \models \gamma$

Outline

Introduction

Propositional logic

Inference rules

Horn and definite clauses

Resolution

Example application

Horn and definite clauses

Horn clause

A Horn clause is either a:

$$P_1 \wedge P_2 \wedge \dots \wedge P_n \Rightarrow Q$$

definite clause

$$Q$$

fact

$$P_1 \wedge P_2 \wedge \dots \wedge P_n \Rightarrow \textit{False}$$

goal clause

(definite clause with $n = 0$)

where $P_1, P_2, \dots, P_n, Q$ are propositional symbols

Horn and definite clauses

Horn clause

A Horn clause is either a:

$$P_1 \wedge P_2 \wedge \dots \wedge P_n \Rightarrow Q$$

definite clause

$$Q$$

fact

$$P_1 \wedge P_2 \wedge \dots \wedge P_n \Rightarrow \textit{False}$$

goal clause

(definite clause with $n = 0$)

where $P_1, P_2, \dots, P_n, Q$ are propositional symbols

If a KB contains only definite clauses, then repeated applications of Modus Ponens is both sound and complete for any fact Q : $KB \models Q \Leftrightarrow KB \vdash_{MPA} Q$

Horn and definite clauses

Horn clause

A Horn clause is either a:

$$P_1 \wedge P_2 \wedge \dots \wedge P_n \Rightarrow Q$$

definite clause

$$Q$$

fact

$$P_1 \wedge P_2 \wedge \dots \wedge P_n \Rightarrow \textit{False}$$

goal clause

(definite clause with $n = 0$)

where $P_1, P_2, \dots, P_n, Q$ are propositional symbols

If a KB contains only definite clauses, then repeated applications of Modus Ponens is both sound and complete for any fact Q : $KB \models Q \Leftrightarrow KB \vdash_{MPA} Q$

Deciding $KB \models Q$ can then also be done in **linear time** in the size of the KB

Definite clauses: example

Review: Modus Ponens

$$\frac{\alpha, \quad \alpha \Rightarrow \beta}{\beta}$$

Given $KB = \{Rain, Weekday, Rain \Rightarrow Wet, Wet \wedge Weekday \Rightarrow Traffic, \\ Traffic \wedge Careless \Rightarrow Accident\}$
does $KB \models Traffic$?

Definite clauses: example

Review: Modus Ponens

$$\frac{\alpha, \quad \alpha \Rightarrow \beta}{\beta}$$

Given $KB = \{Rain, Weekday, Rain \Rightarrow Wet, Wet \wedge Weekday \Rightarrow Traffic, \\ Traffic \wedge Careless \Rightarrow Accident\}$

does $KB \models Traffic$?

We can check if $KB \vdash_{MPA} Traffic$:

Definite clauses: example

Review: Modus Ponens

$$\frac{\alpha, \quad \alpha \Rightarrow \beta}{\beta}$$

Given $KB = \{Rain, Weekday, Rain \Rightarrow Wet, Wet \wedge Weekday \Rightarrow Traffic, \\ Traffic \wedge Careless \Rightarrow Accident\}$

does $KB \models Traffic$?

We can check if $KB \vdash_{MPA} Traffic$:

Apply Modus Ponens to $Rain$ and $Rain \Rightarrow Wet$ to derive Wet :

$KB = \{Rain, Weekday, Rain \Rightarrow Wet, Wet \wedge Weekday \Rightarrow Traffic, \\ Traffic \wedge Careless \Rightarrow Accident, Wet\}$

Definite clauses: example

Review: Modus Ponens

$$\frac{\alpha, \quad \alpha \Rightarrow \beta}{\beta}$$

Given $KB = \{Rain, Weekday, Rain \Rightarrow Wet, Wet \wedge Weekday \Rightarrow Traffic, \\ Traffic \wedge Careless \Rightarrow Accident\}$

does $KB \models Traffic$?

We can check if $KB \vdash_{MPA} Traffic$:

Apply Modus Ponens to $Rain$ and $Rain \Rightarrow Wet$ to derive Wet :

$KB = \{Rain, Weekday, Rain \Rightarrow Wet, Wet \wedge Weekday \Rightarrow Traffic, \\ Traffic \wedge Careless \Rightarrow Accident, Wet\}$

Apply Modus Ponens to $Wet \wedge Weekday$ and $Wet \wedge Weekday \Rightarrow Traffic$ to derive $Traffic$

Answer: yes, $KB \vdash_{MPA} Traffic$, therefore $KB \models Traffic$

Horn clauses: significance (not curriculum)

- The programming language Prolog uses only Horn clauses
 - Horn clauses in first-order logic (topic next week) is Turing-complete
- BusTUC was programmed in Prolog using first-order logic
- Example newer software written mostly in Prolog: [TerminusDB](#)

Prolog example, where :- means $\Leftarrow$, and $\alpha \Rightarrow \beta \equiv \beta \Leftarrow \alpha$:

```
% Filename: rain.prolog

consult(user). % Start of KB

rain.          % It is raining
wet :- rain.    % If it is raining, it is wet
slippery :- wet. % If it is wet, it is slippery

end_of_file. % End of KB

slippery. % Is it slippery? Output: true

% Run for example with: swipl < rain.prolog
```

Definite clause inference algorithms

Inference algorithms for deciding $KB \models Q$, when KB contains only definite clauses:

- Forward chaining
 - Derives new facts until Q is derived
 - Shown in Figure 7.15 in the book
- Backward chaining
 - Works backward from Q and attempts to derive the premises (e.g. P_1, P_2) of the implications (e.g. $P_1 \wedge P_2 \Rightarrow Q$) with Q as conclusion. If the premises are all *true*, then Q must also be *true*

Outline

Introduction

Propositional logic

Inference rules

Horn and definite clauses

Resolution

Example application

Resolution inference algorithm

The resolution inference algorithm (RA) is both sound and complete for all propositional logic sentences:

$$KB \models \alpha \Leftrightarrow KB \vdash_{RA} \alpha$$

Resolution inference algorithm

The resolution inference algorithm (RA) is both sound and complete for all propositional logic sentences:

$$KB \models \alpha \Leftrightarrow KB \vdash_{RA} \alpha$$

However, it runs in exponential time in the size of the KB

Resolution inference rule

Literal

A literal is a propositional symbol or a negated propositional symbol, e.g. $\neg P$, Q

Resolution inference rule

Literal

A literal is a propositional symbol or a negated propositional symbol, e.g. $\neg P$, Q

Clause

A clause is a disjunction of literals, e.g. $\neg P \vee Q \vee \neg R \vee S \vee \neg T \vee U$

Resolution inference rule

Literal

A literal is a propositional symbol or a negated propositional symbol, e.g. $\neg P$, Q

Clause

A clause is a disjunction of literals, e.g. $\neg P \vee Q \vee \neg R \vee S \vee \neg T \vee U$

Resolution inference rule

$$\frac{a_1 \vee a_2 \vee \dots \vee \textcolor{red}{c}, \quad \neg \textcolor{red}{c} \vee b_1 \vee b_2 \vee \dots}{a_1 \vee a_2 \vee \dots \vee b_1 \vee b_2 \vee \dots}$$

where $a_1, a_2, \dots, b_1, b_2, \dots, \textcolor{red}{c}$ are literals, i.e. if $c = \neg P$, then $\neg c = \neg(\neg P) = P$

For example:

$$\frac{\textit{Rain} \vee \textcolor{red}{Snow}, \quad \neg \textcolor{red}{Snow} \vee \textit{Traffic}}{\textit{Rain} \vee \textit{Traffic}}$$

Note that Modus Ponens is a special case of the resolution inference rule, e.g.: $\frac{\textcolor{red}{Rain}, \quad \neg \textcolor{red}{Rain} \vee \textit{Wet}}{\textit{Wet}}$

Conjunctive normal form (CNF)

The resolution rule only applies to clauses. Answer: convert to conjunctive normal form

Conjunctive normal form (CNF)

The resolution rule only applies to clauses. Answer: convert to conjunctive normal form

Conjunctive normal form

A conjunctive normal form is a conjunction of clauses, e.g. $(P \vee Q \vee \neg R) \wedge (\neg S \vee T)$

Conjunctive normal form (CNF)

The resolution rule only applies to clauses. Answer: convert to conjunctive normal form

Conjunctive normal form

A conjunctive normal form is a conjunction of clauses, e.g. $(P \vee Q \vee \neg R) \wedge (\neg S \vee T)$

Conversion to conjunctive normal form

Any sentence of propositional logic can be converted into an equivalent sentence in conjunctive normal form

Algorithm:

1. Eliminate $\Leftrightarrow$: $\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}$
2. Eliminate $\Rightarrow$: $\frac{\alpha \Rightarrow \beta}{\neg \alpha \vee \beta}$
3. Repeatedly move $\neg$ inwards and eliminate double negation: $\frac{\neg(\alpha \wedge \beta)}{\neg \alpha \vee \neg \beta}$ $\frac{\neg(\alpha \vee \beta)}{\neg \alpha \wedge \neg \beta}$ $\frac{\neg(\neg \alpha)}{\alpha}$
4. Distribute $\vee$ over $\wedge$: $\frac{\alpha \vee (\beta \wedge \gamma)}{(\alpha \vee \beta) \wedge (\alpha \vee \gamma)}$ $\frac{(\alpha \wedge \beta) \vee \gamma}{(\alpha \vee \gamma) \wedge (\beta \vee \gamma)}$

Resolution inference algorithm

Resolution inference algorithm

1. Add $\neg\alpha$ to KB , and convert all KB sentences to conjunctive normal form
2. Separate conjunctions into clauses, e.g. $\{(P \vee Q) \wedge R\}$ becomes $\{P \vee Q, R\}$
3. Repeatedly apply the resolution inference rule, and return unsatisfiable ($KB \models \alpha$) if empty clause (contradiction) is derived, e.g. $\frac{P, \neg P}{\text{empty clause}}$, otherwise return satisfiable ($KB \not\models \alpha$)

Resolution inference algorithm: example

Given $KB = \{P, P \vee Q \Rightarrow R\}$, does $KB \models R$?

Resolution inference algorithm: example

Given $KB = \{P, P \vee Q \Rightarrow R\}$, does $KB \models R$?

Add $\neg R$ to KB :

$$KB \leftarrow \{P, P \vee Q \Rightarrow R, \neg R\}$$

Resolution inference algorithm: example

Given $KB = \{P, P \vee Q \Rightarrow R\}$, does $KB \models R$?

Add $\neg R$ to KB : $KB \leftarrow \{P, P \vee Q \Rightarrow R, \neg R\}$

Convert sentences to conjunctive normal form:

Resolution inference algorithm: example

Given $KB = \{P, P \vee Q \Rightarrow R\}$, does $KB \models R$?

Add $\neg R$ to KB : $KB \leftarrow \{P, P \vee Q \Rightarrow R, \neg R\}$

Convert sentences to conjunctive normal form:

Eliminate $\Rightarrow$: $KB \leftarrow \{P, \neg(P \vee Q) \vee R, \neg R\}$

Resolution inference algorithm: example

Given $KB = \{P, P \vee Q \Rightarrow R\}$, does $KB \models R$?

Add $\neg R$ to KB : $KB \leftarrow \{P, P \vee Q \Rightarrow R, \neg R\}$

Convert sentences to conjunctive normal form:

Eliminate $\Rightarrow$: $KB \leftarrow \{P, \neg(P \vee Q) \vee R, \neg R\}$

Move $\neg$ inwards: $KB \leftarrow \{P, (\neg P \wedge \neg Q) \vee R, \neg R\}$

Resolution inference algorithm: example

Given $KB = \{P, P \vee Q \Rightarrow R\}$, does $KB \models R$?

Add $\neg R$ to KB : $KB \leftarrow \{P, P \vee Q \Rightarrow R, \neg R\}$

Convert sentences to conjunctive normal form:

Eliminate $\Rightarrow$: $KB \leftarrow \{P, \neg(P \vee Q) \vee R, \neg R\}$

Move $\neg$ inwards: $KB \leftarrow \{P, (\neg P \wedge \neg Q) \vee R, \neg R\}$

Distribute $\vee$ over $\wedge$: $KB \leftarrow \{P, (\neg P \vee R) \wedge (\neg Q \vee R), \neg R\}$

Resolution inference algorithm: example

Given $KB = \{P, P \vee Q \Rightarrow R\}$, does $KB \models R$?

Add $\neg R$ to KB : $KB \leftarrow \{P, P \vee Q \Rightarrow R, \neg R\}$

Convert sentences to conjunctive normal form:

Eliminate $\Rightarrow$: $KB \leftarrow \{P, \neg(P \vee Q) \vee R, \neg R\}$

Move $\neg$ inwards: $KB \leftarrow \{P, (\neg P \wedge \neg Q) \vee R, \neg R\}$

Distribute $\vee$ over $\wedge$: $KB \leftarrow \{P, (\neg P \vee R) \wedge (\neg Q \vee R), \neg R\}$

Separate conjunctions into clauses: $KB \leftarrow \{P, \neg P \vee R, \neg Q \vee R, \neg R\}$

Resolution inference algorithm: example

Given $KB = \{P, P \vee Q \Rightarrow R\}$, does $KB \models R$?

Add $\neg R$ to KB : $KB \leftarrow \{P, P \vee Q \Rightarrow R, \neg R\}$

Convert sentences to conjunctive normal form:

Eliminate $\Rightarrow$: $KB \leftarrow \{P, \neg(P \vee Q) \vee R, \neg R\}$

Move $\neg$ inwards: $KB \leftarrow \{P, (\neg P \wedge \neg Q) \vee R, \neg R\}$

Distribute $\vee$ over $\wedge$: $KB \leftarrow \{P, (\neg P \vee R) \wedge (\neg Q \vee R), \neg R\}$

Separate conjunctions into clauses: $KB \leftarrow \{P, \neg P \vee R, \neg Q \vee R, \neg R\}$

Repeatedly apply the resolution inference rule:

Resolution inference algorithm: example

Given $KB = \{P, P \vee Q \Rightarrow R\}$, does $KB \models R$?

Add $\neg R$ to KB :

$$KB \leftarrow \{P, P \vee Q \Rightarrow R, \neg R\}$$

Convert sentences to conjunctive normal form:

Eliminate $\Rightarrow$:

$$KB \leftarrow \{P, \neg(P \vee Q) \vee R, \neg R\}$$

Move $\neg$ inwards:

$$KB \leftarrow \{P, (\neg P \wedge \neg Q) \vee R, \neg R\}$$

Distribute $\vee$ over $\wedge$:

$$KB \leftarrow \{P, (\neg P \vee R) \wedge (\neg Q \vee R), \neg R\}$$

Separate conjunctions into clauses:

$$KB \leftarrow \{P, \neg P \vee R, \neg Q \vee R, \neg R\}$$

Repeatedly apply the resolution inference rule:

$$KB \leftarrow \{P, \neg P \vee R, \neg Q \vee R, \neg R, R\}$$

Resolution inference algorithm: example

Given $KB = \{P, P \vee Q \Rightarrow R\}$, does $KB \models R$?

Add $\neg R$ to KB :

$$KB \leftarrow \{P, P \vee Q \Rightarrow R, \neg R\}$$

Convert sentences to conjunctive normal form:

Eliminate $\Rightarrow$:

$$KB \leftarrow \{P, \neg(P \vee Q) \vee R, \neg R\}$$

Move $\neg$ inwards:

$$KB \leftarrow \{P, (\neg P \wedge \neg Q) \vee R, \neg R\}$$

Distribute $\vee$ over $\wedge$:

$$KB \leftarrow \{P, (\neg P \vee R) \wedge (\neg Q \vee R), \neg R\}$$

Separate conjunctions into clauses:

$$KB \leftarrow \{P, \neg P \vee R, \neg Q \vee R, \neg R\}$$

Repeatedly apply the resolution inference rule:

$$KB \leftarrow \{P, \neg P \vee R, \neg Q \vee R, \neg R, R\}$$

$$KB \leftarrow \{P, \neg P \vee R, \neg Q \vee R, \neg R, R, \neg Q\}$$

Resolution inference algorithm: example

Given $KB = \{P, P \vee Q \Rightarrow R\}$, does $KB \models R$?

Add $\neg R$ to KB :

$$KB \leftarrow \{P, P \vee Q \Rightarrow R, \neg R\}$$

Convert sentences to conjunctive normal form:

Eliminate $\Rightarrow$:

$$KB \leftarrow \{P, \neg(P \vee Q) \vee R, \neg R\}$$

Move $\neg$ inwards:

$$KB \leftarrow \{P, (\neg P \wedge \neg Q) \vee R, \neg R\}$$

Distribute $\vee$ over $\wedge$:

$$KB \leftarrow \{P, (\neg P \vee R) \wedge (\neg Q \vee R), \neg R\}$$

Separate conjunctions into clauses:

$$KB \leftarrow \{P, \neg P \vee R, \neg Q \vee R, \neg R\}$$

Repeatedly apply the resolution inference rule:

$$KB \leftarrow \{P, \neg P \vee R, \neg Q \vee R, \neg R, R\}$$

$$KB \leftarrow \{P, \neg P \vee R, \neg Q \vee R, \neg R, R, \neg Q\}$$

An empty clause is derived:

$$KB \leftarrow \{P, \neg P \vee R, \neg Q \vee R, \neg R, R, \neg Q\}$$

Answer: $KB \models R$

Resolution inference algorithm: heuristics

Example derivations when deciding $KB \models R$ without heuristics:

Initial $KB \cup \{\neg R\}$ in conjunctive normal form

$$P$$

$$\neg P \vee R$$

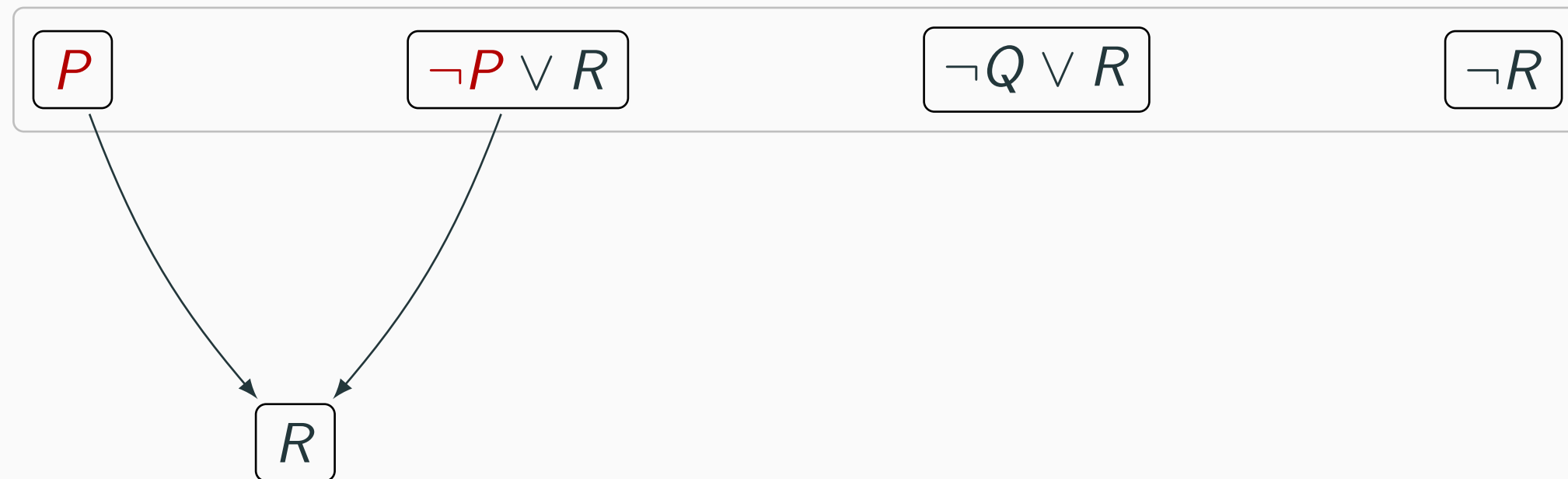
$$\neg Q \vee R$$

$$\neg R$$

Resolution inference algorithm: heuristics

Example derivations when deciding $KB \models R$ without heuristics:

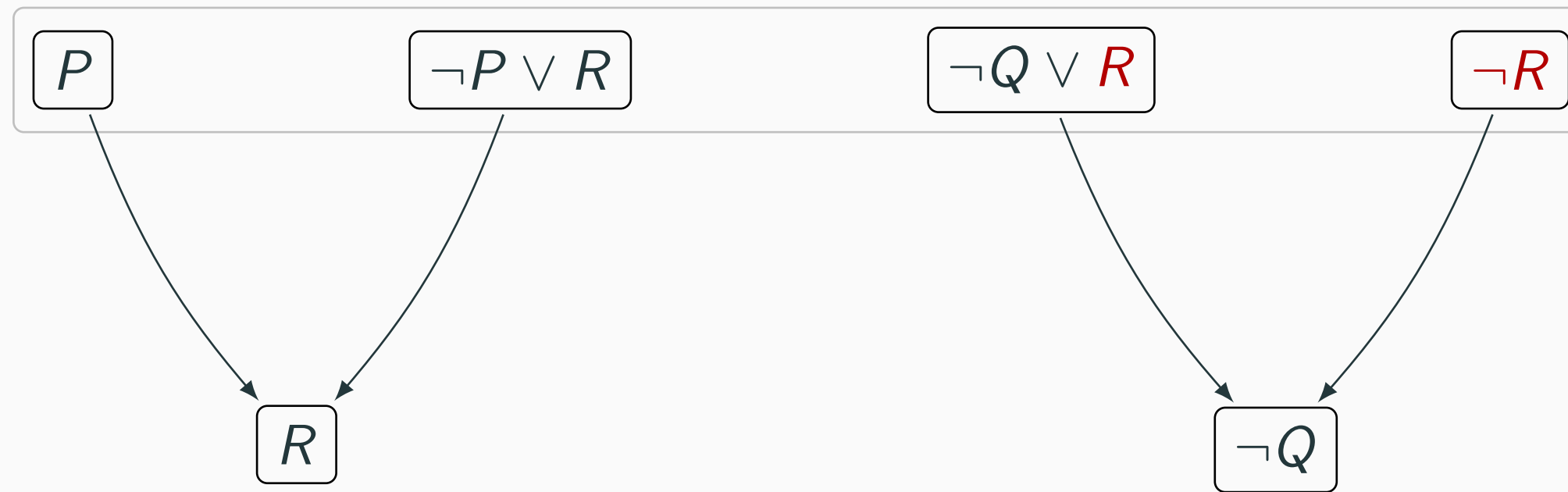
Initial $KB \cup \{\neg R\}$ in conjunctive normal form



Resolution inference algorithm: heuristics

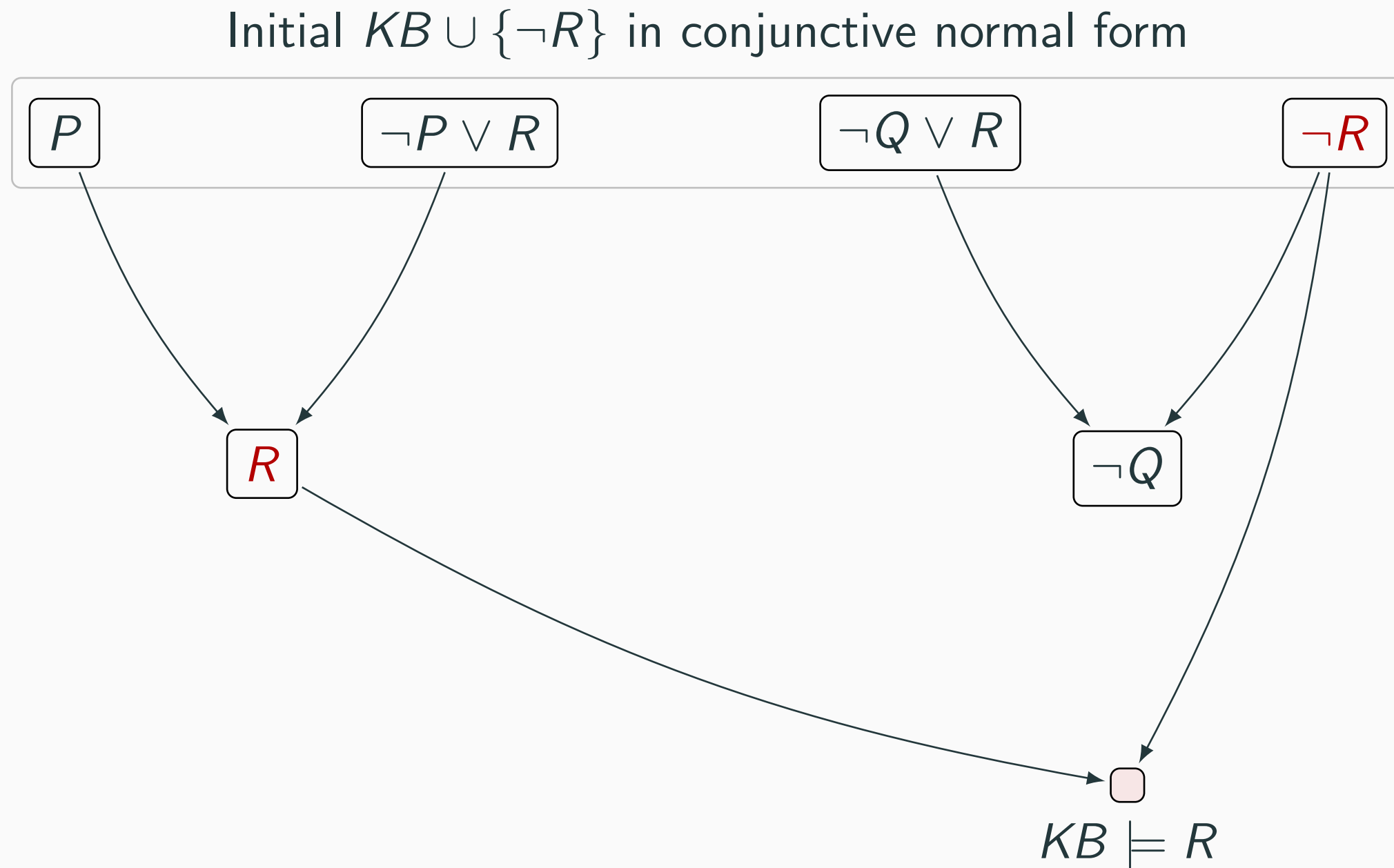
Example derivations when deciding $KB \models R$ without heuristics:

Initial $KB \cup \{\neg R\}$ in conjunctive normal form



Resolution inference algorithm: heuristics

Example derivations when deciding $KB \models R$ without heuristics:



Resolution inference algorithm: heuristics

Example derivations when deciding $KB \models R$ with the unit preference heuristic (p. 326):

Initial $KB \cup \{\neg R\}$ in conjunctive normal form

$$P$$

$$\neg P \vee R$$

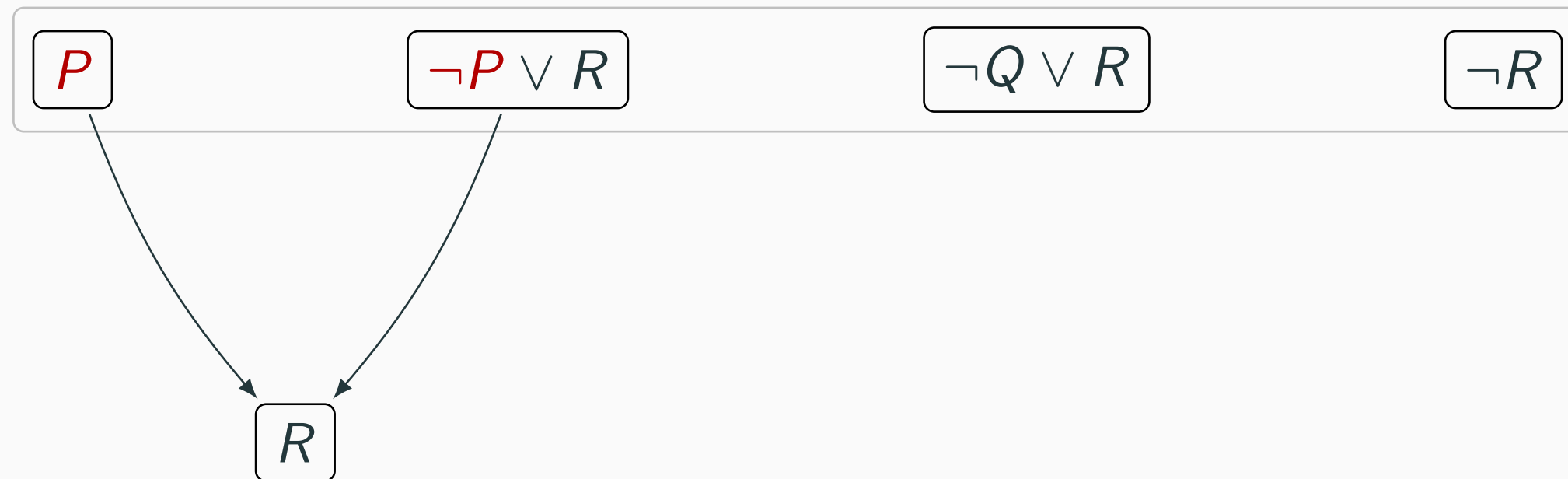
$$\neg Q \vee R$$

$$\neg R$$

Resolution inference algorithm: heuristics

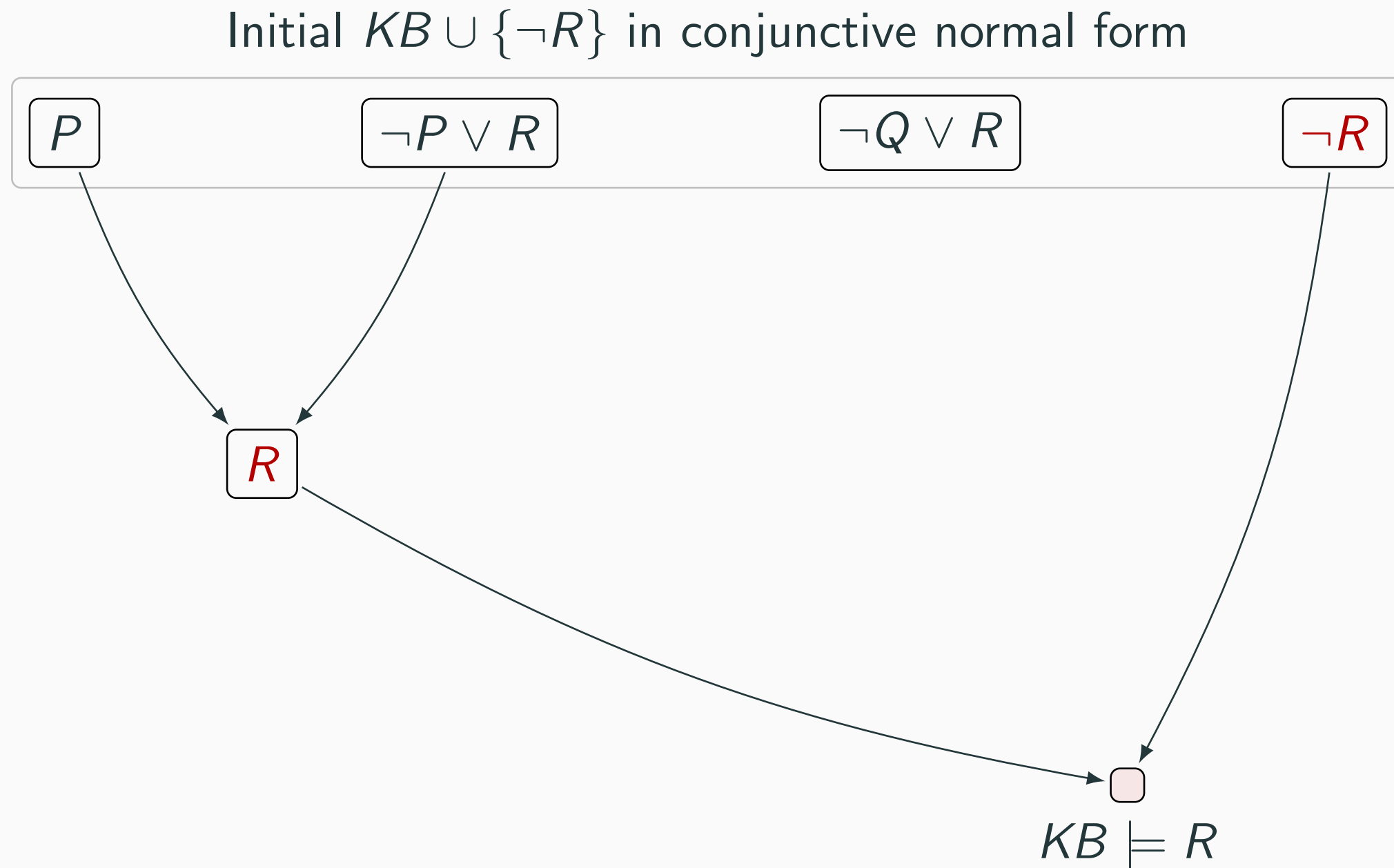
Example derivations when deciding $KB \models R$ with the unit preference heuristic (p. 326):

Initial $KB \cup \{\neg R\}$ in conjunctive normal form



Resolution inference algorithm: heuristics

Example derivations when deciding $KB \models R$ with the unit preference heuristic (p. 326):



Outline

Introduction

Propositional logic

Inference rules

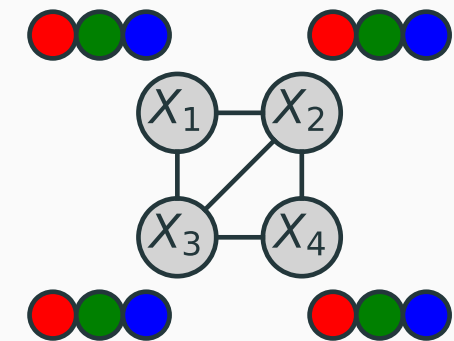
Horn and definite clauses

Resolution

Example application

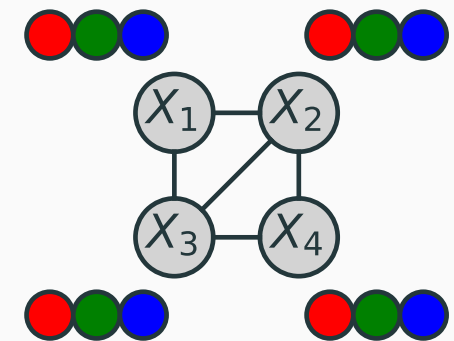
Solving CSPs using repeated applications of the resolution inference rule

$$\begin{aligned} KB = \{ & (X1r \wedge \neg X1g \wedge \neg X1b) \vee (\neg X1r \wedge X1g \wedge \neg X1b) \vee (\neg X1r \wedge \neg X1g \wedge X1b), \\ & (X2r \wedge \neg X2g \wedge \neg X2b) \vee (\neg X2r \wedge X2g \wedge \neg X2b) \vee (\neg X2r \wedge \neg X2g \wedge X2b), \\ & (X3r \wedge \neg X3g \wedge \neg X3b) \vee (\neg X3r \wedge X3g \wedge \neg X3b) \vee (\neg X3r \wedge \neg X3g \wedge X3b), \\ & (X4r \wedge \neg X4g \wedge \neg X4b) \vee (\neg X4r \wedge X4g \wedge \neg X4b) \vee (\neg X4r \wedge \neg X4g \wedge X4b), \\ & \neg(X1r \wedge X2r) \wedge \neg(X1g \wedge X2g) \wedge \neg(X1b \wedge X2b), \\ & \neg(X1r \wedge X3r) \wedge \neg(X1g \wedge X3g) \wedge \neg(X1b \wedge X3b), \\ & \neg(X2r \wedge X3r) \wedge \neg(X2g \wedge X3g) \wedge \neg(X2b \wedge X3b), \\ & \neg(X2r \wedge X4r) \wedge \neg(X2g \wedge X4g) \wedge \neg(X2b \wedge X4b), \\ & \neg(X3r \wedge X4r) \wedge \neg(X3g \wedge X4g) \wedge \neg(X3b \wedge X4b), \\ & \} \end{aligned}$$



Solving CSPs using repeated applications of the resolution inference rule

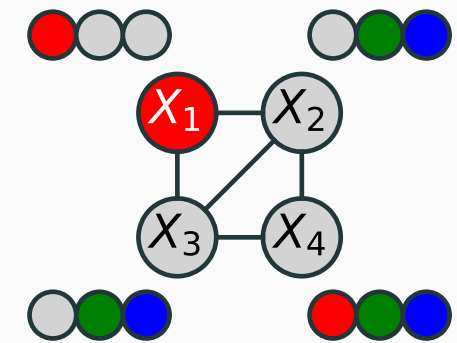
$$\begin{aligned} KB = \{ & (X1r \wedge \neg X1g \wedge \neg X1b) \vee (\neg X1r \wedge X1g \wedge \neg X1b) \vee (\neg X1r \wedge \neg X1g \wedge X1b), \\ & (X2r \wedge \neg X2g \wedge \neg X2b) \vee (\neg X2r \wedge X2g \wedge \neg X2b) \vee (\neg X2r \wedge \neg X2g \wedge X2b), \\ & (X3r \wedge \neg X3g \wedge \neg X3b) \vee (\neg X3r \wedge X3g \wedge \neg X3b) \vee (\neg X3r \wedge \neg X3g \wedge X3b), \\ & (X4r \wedge \neg X4g \wedge \neg X4b) \vee (\neg X4r \wedge X4g \wedge \neg X4b) \vee (\neg X4r \wedge \neg X4g \wedge X4b), \\ & \neg(X1r \wedge X2r) \wedge \neg(X1g \wedge X2g) \wedge \neg(X1b \wedge X2b), \\ & \neg(X1r \wedge X3r) \wedge \neg(X1g \wedge X3g) \wedge \neg(X1b \wedge X3b), \\ & \neg(X2r \wedge X3r) \wedge \neg(X2g \wedge X3g) \wedge \neg(X2b \wedge X3b), \\ & \neg(X2r \wedge X4r) \wedge \neg(X2g \wedge X4g) \wedge \neg(X2b \wedge X4b), \\ & \neg(X3r \wedge X4r) \wedge \neg(X3g \wedge X4g) \wedge \neg(X3b \wedge X4b), \\ & \} \end{aligned}$$



$$M(KB \cup \{X1r\}) = \emptyset?$$

Solving CSPs using repeated applications of the resolution inference rule

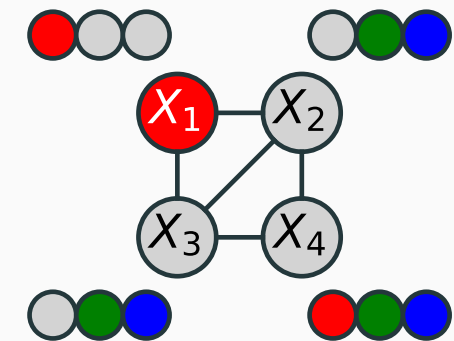
$$\begin{aligned}
 KB = \{ & (X1r \wedge \neg X1g \wedge \neg X1b) \vee (\neg X1r \wedge X1g \wedge \neg X1b) \vee (\neg X1r \wedge \neg X1g \wedge X1b), \\
 & (X2r \wedge \neg X2g \wedge \neg X2b) \vee (\neg X2r \wedge X2g \wedge \neg X2b) \vee (\neg X2r \wedge \neg X2g \wedge X2b), \\
 & (X3r \wedge \neg X3g \wedge \neg X3b) \vee (\neg X3r \wedge X3g \wedge \neg X3b) \vee (\neg X3r \wedge \neg X3g \wedge X3b), \\
 & (X4r \wedge \neg X4g \wedge \neg X4b) \vee (\neg X4r \wedge X4g \wedge \neg X4b) \vee (\neg X4r \wedge \neg X4g \wedge X4b), \\
 & \neg(X1r \wedge X2r) \wedge \neg(X1g \wedge X2g) \wedge \neg(X1b \wedge X2b), \\
 & \neg(X1r \wedge X3r) \wedge \neg(X1g \wedge X3g) \wedge \neg(X1b \wedge X3b), \\
 & \neg(X2r \wedge X3r) \wedge \neg(X2g \wedge X3g) \wedge \neg(X2b \wedge X3b), \\
 & \neg(X2r \wedge X4r) \wedge \neg(X2g \wedge X4g) \wedge \neg(X2b \wedge X4b), \\
 & \neg(X3r \wedge X4r) \wedge \neg(X3g \wedge X4g) \wedge \neg(X3b \wedge X4b), \\
 & X1r, \neg X1g, \neg X1b, \neg X2r, \neg X3r \\
 & \left. \vphantom{KB = \{}} \right\}
 \end{aligned}$$



$M(KB \cup \{X1r\}) = \emptyset?$ no: $KB \leftarrow KB \cup \{X1r, \neg X1g, \neg X1b, \neg X2r, \neg X3r\}$
 $(\neg X1g, \neg X1b, \neg X2r, \neg X3r \text{ are also derived})$

Solving CSPs using repeated applications of the resolution inference rule

$$\begin{aligned}
 KB = \{ & (X1r \wedge \neg X1g \wedge \neg X1b) \vee (\neg X1r \wedge X1g \wedge \neg X1b) \vee (\neg X1r \wedge \neg X1g \wedge X1b), \\
 & (X2r \wedge \neg X2g \wedge \neg X2b) \vee (\neg X2r \wedge X2g \wedge \neg X2b) \vee (\neg X2r \wedge \neg X2g \wedge X2b), \\
 & (X3r \wedge \neg X3g \wedge \neg X3b) \vee (\neg X3r \wedge X3g \wedge \neg X3b) \vee (\neg X3r \wedge \neg X3g \wedge X3b), \\
 & (X4r \wedge \neg X4g \wedge \neg X4b) \vee (\neg X4r \wedge X4g \wedge \neg X4b) \vee (\neg X4r \wedge \neg X4g \wedge X4b), \\
 & \neg(X1r \wedge X2r) \wedge \neg(X1g \wedge X2g) \wedge \neg(X1b \wedge X2b), \\
 & \neg(X1r \wedge X3r) \wedge \neg(X1g \wedge X3g) \wedge \neg(X1b \wedge X3b), \\
 & \neg(X2r \wedge X3r) \wedge \neg(X2g \wedge X3g) \wedge \neg(X2b \wedge X3b), \\
 & \neg(X2r \wedge X4r) \wedge \neg(X2g \wedge X4g) \wedge \neg(X2b \wedge X4b), \\
 & \neg(X3r \wedge X4r) \wedge \neg(X3g \wedge X4g) \wedge \neg(X3b \wedge X4b), \\
 & X1r, \neg X1g, \neg X1b, \neg X2r, \neg X3r \\
 & \left. \vphantom{KB = \{}} \right\}
 \end{aligned}$$

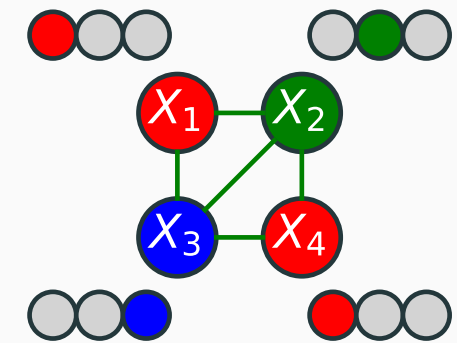


$M(KB \cup \{X1r\}) = \emptyset?$ no: $KB \leftarrow KB \cup \{X1r, \neg X1g, \neg X1b, \neg X2r, \neg X3r\}$
 $(\neg X1g, \neg X1b, \neg X2r, \neg X3r \text{ are also derived})$

$M(KB \cup \{X2g\}) = \emptyset?$

Solving CSPs using repeated applications of the resolution inference rule

$$\begin{aligned}
 KB = \{ & (X1r \wedge \neg X1g \wedge \neg X1b) \vee (\neg X1r \wedge X1g \wedge \neg X1b) \vee (\neg X1r \wedge \neg X1g \wedge X1b), \\
 & (X2r \wedge \neg X2g \wedge \neg X2b) \vee (\neg X2r \wedge X2g \wedge \neg X2b) \vee (\neg X2r \wedge \neg X2g \wedge X2b), \\
 & (X3r \wedge \neg X3g \wedge \neg X3b) \vee (\neg X3r \wedge X3g \wedge \neg X3b) \vee (\neg X3r \wedge \neg X3g \wedge X3b), \\
 & (X4r \wedge \neg X4g \wedge \neg X4b) \vee (\neg X4r \wedge X4g \wedge \neg X4b) \vee (\neg X4r \wedge \neg X4g \wedge X4b), \\
 & \neg(X1r \wedge X2r) \wedge \neg(X1g \wedge X2g) \wedge \neg(X1b \wedge X2b), \\
 & \neg(X1r \wedge X3r) \wedge \neg(X1g \wedge X3g) \wedge \neg(X1b \wedge X3b), \\
 & \neg(X2r \wedge X3r) \wedge \neg(X2g \wedge X3g) \wedge \neg(X2b \wedge X3b), \\
 & \neg(X2r \wedge X4r) \wedge \neg(X2g \wedge X4g) \wedge \neg(X2b \wedge X4b), \\
 & \neg(X3r \wedge X4r) \wedge \neg(X3g \wedge X4g) \wedge \neg(X3b \wedge X4b), \\
 & X1r, \neg X1g, \neg X1b, \neg X2r, \neg X3r, X2g, \neg X2b, \neg X3g, X3b, X4r, \neg X4g, \neg X4b \}
 \end{aligned}$$



$$\begin{aligned}
 M(KB \cup \{X1r\}) = \emptyset? \text{ no: } KB \leftarrow KB \cup \{X1r, \neg X1g, \neg X1b, \neg X2r, \neg X3r\} \\
 (\neg X1g, \neg X1b, \neg X2r, \neg X3r \text{ are also derived})
 \end{aligned}$$

$$\begin{aligned}
 M(KB \cup \{X2g\}) = \emptyset? \text{ no: } KB \leftarrow KB \cup \{X2g, \neg X2b, \neg X3g, X3b, X4r, \neg X4g, \neg X4b\} \\
 (\neg X2b, \neg X3g, X3b, X4r, \neg X4g, \neg X4b \text{ are also derived})
 \end{aligned}$$